

PCA Esparso

Profs. Alexandre Salles da Cunha e Ana Paula Couto

Universidade Federal de Minas Gerais
Departamento de Ciência da Computação
Belo Horizonte, Brasil

`acunha@dcc.ufmg.br`, `ana.couto@dcc.ufmg.br`

2026/1



UNIVERSIDADE FEDERAL
DE MINAS GERAIS



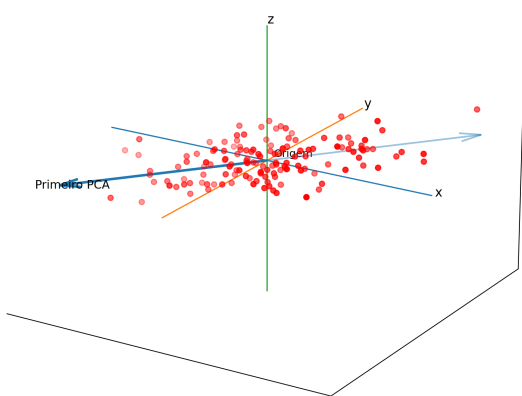


Figura: Ilustração do PCA clássico.

- PCA clássico (1ª componente):

$$\max_{\|x\|_2=1} x^\top Qx$$

- Problema: x costuma ser *denso* $\Rightarrow$ baixa interpretabilidade e sem seleção de variáveis.
- Sparse PCA (SPCA): impor esparsidade k (apenas k coordenadas não nulas):

$$\max_x x^\top Qx \quad \text{s.a.} \quad \|x\|_2 = 1, \quad \|x\|_0 \leq k.$$

Objetivo

Por meio de Sparse PCA (com $k = 6$) identificar tópicos principais em uma grande base de textos.

- 1 300.000 artigos do *New York Times* disponíveis na UCI Machine Learning Repository. Dicionário de 102.660 palavras. Dados não rotulados.
- 2 Representação vetorial dos documentos: *bag-of-words*.
- 3 Cinco primeiros PCAs esparsos (heurística e remoção de termos)

PCAE1	PCAE2	PCAE3	PCAE4	PCAE5
million percent business company market companies	point play team season game	official government united_states u_s attach	president campaing bush administration	school program children student

Idéia central do modelo: Vamos usar uma variável de decisão adicional, $y \in \{0, 1\}^n$, que controla quais features podem participar do vetor (PCA esperso) desejado. Se $y_i = 1$, podemos ter $x_i \neq 0$, caso contrário, isto é, $y_i = 0 \rightarrow x_i = 0$.

$$\begin{aligned} \max_{x,y} \quad & x^\top Q x \\ \text{s.a.} \quad & \sum_{i=1}^n x_i^2 = 1 \\ & -y_i \leq x_i \leq y_i, \quad i = 1, \dots, n \\ & \sum_{i=1}^n y_i = k, \quad y \in \{0, 1\}^n. \end{aligned}$$

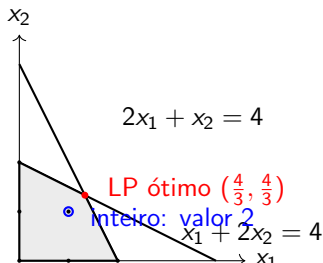
Abordagem vale para outras classes de problemas de otimização, não convexos.

- Queremos resolver um problema **discreto** (ex.: variáveis inteiras/binárias) com garantia de otimalidade.
- **Branch**: particionar o espaço de soluções em subproblemas menores.
- **Bound**: em cada nó, resolver uma **relaxação** mais fácil $\Rightarrow$ produz um limite (dual) para o valor ótimo da função objetivo daquele nó.
- **Poda (pruning)**: se o limite dual garante que o nó não pode melhorar a melhor solução inteira já encontrada (incumbent, que produz um limite primal válido), descartamos toda a subárvore enraizada naquele nó. Isto se chama poda.
- A poda também pode ocorrer por **inviabilidade**.

B&B = *enumerar inteligentemente usando limites duais e primais para não enumerar quase nada.*

$$\max x_1 + x_2 \quad \text{s.a.} \quad \begin{cases} x_1 + 2x_2 \leq 4 \\ 2x_1 + x_2 \leq 4 \\ x_1, x_2 \in \mathbb{Z}_{\geq 0} \end{cases}$$

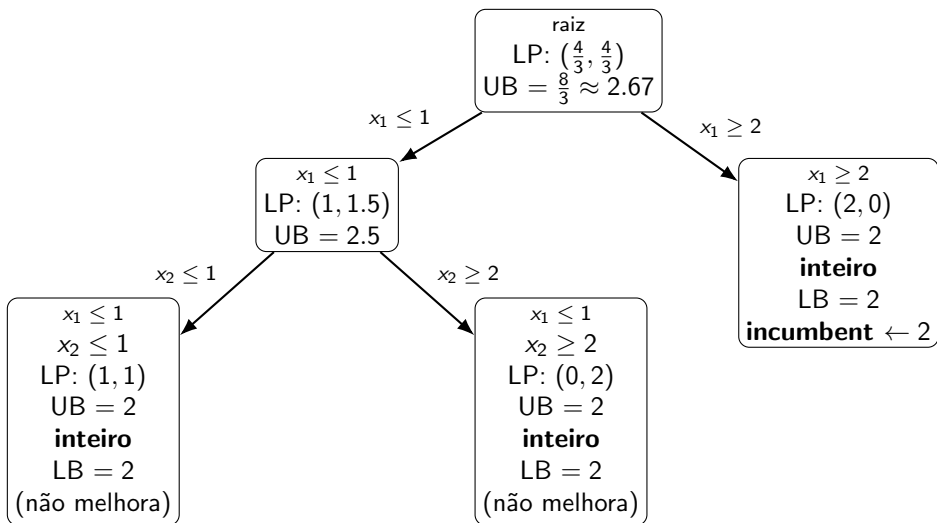
- Relaxação de Programação Linear (LP, obtida relaxando a integralidade) quando admite um ótimo, o admite em um vértice do poliedro.
- Aqui, o ótimo do LP é **fracionário**: $(x_1, x_2) = (\frac{4}{3}, \frac{4}{3})$, valor = $\frac{8}{3}$.
- O valor ótimo da relaxação LP produz um limitante dual, **upper bound**; mas precisamos de solução **inteira**.



Branching e bounds: árvore B&B (exemplo)

Ramificamos em x_1 (do LP: $x_1 = \frac{4}{3}$):

$$x_1 \leq 1 \quad \text{ou} \quad x_1 \geq 2.$$



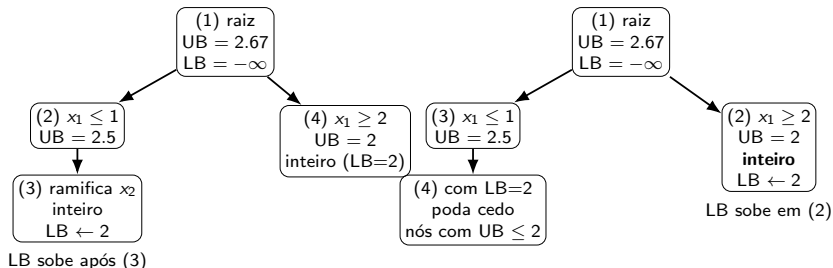
Com incumbent = 2, todo nó com $UB \leq 2$ pode ser podado.

A ordem de exploração afeta quando o LB sobe

O **LB (bound primal)** só aumenta quando encontramos uma solução **inteira** melhor. A ordem dos nós visitados determina *quando* isso acontece.

DFS: esquerda primeiro
(LB sobe mais tarde)

Direita primeiro
(LB sobe cedo)



Encontrar um bom incumbent cedo (LB alto) acelera a poda e reduz a árvore. A estratégia de investigação afeta como os limites duais e primais são aprimorados.

Ideias centrais do algoritmo (maximização)

- ❶ Inicialize **incumbent** (melhor solução inteira) com $-\infty$ e crie o nó raiz.
 - ❷ Enquanto houver nós ativos:
 - ❶ Selecione um nó (ex.: maior UB = *best-bound*).
 - ❷ Resolva a **relaxação** $\Rightarrow$ obtenha UB e uma solução fracionária.
 - ❸ Se a Relaxação for inviável, **poda**.
 - ❹ Se a solução for inteira: atualize incumbent.
 - ❺ Se $UB \leq \text{incumbent}$: **poda**.
 - ❻ Senão: **branch** em alguma variável fracionária e crie filhos.
- **Branching rule**: qual variável ramificar? (mais fracionária, maior impacto, etc.)
 - **Node selection**: DFS (memória baixa) vs best-bound (poda agressiva).
 - **Boa solução inicial** (heurística) melhora muito a poda.

Cada nó é identificado por $(\ell, u) \in \{0, 1\}^n \times \{0, 1\}^n$ impondo

$$\ell \leq y \leq u,$$

onde $\ell_i = 1$ força $y_i = 1$ e $u_i = 0$ força $y_i = 0$.

Problema no nó (SPCA-PDS):

$$\max_{x,y} x^\top Qx$$

$$\text{s.a. } \sum_{i=1}^n x_i^2 = 1, \quad -y_i \leq x_i \leq y_i$$

$$\sum_{i=1}^n y_i = k, \quad y \in \{0, 1\}^n, \quad \ell \leq y \leq u.$$

$\Rightarrow$ Para o nó raiz, temos $\ell = 0_n, u = 1_n$.

- Se enumerássemos todos as possíveis combinações de estados do vetor y (de início sem nos preocuparmos com a restrição $\|y\|_0 \leq k$), teríamos $l = u$ para as todas as folhas na árvore de enumeração completa.
- Desejamos evitar a enumeração completa. Por isso vamos:
 - usar limitantes primais e duais para podar.
 - Não vamos perder tempo com nós (l, u) para os quais já enumeramos mais do que o necessário.
- Quando $l = u$, o nó (l, u) está completamente determinado e a variável y se torna uma constante.
Neste caso, basta removermos de Q as colunas e linhas para as quais $l_i = u_i = 0$, obtendo a matriz Q_u , e o PDS torna-se um PCA clássico, que sabemos resolver:

$$\max_{x, \|x\|_2=1} x_u^T Q_u x_u.$$

- A solução de

$$\max_{x, \|x\|_2=1} x_u^T Q_u x_u$$

é o autovetor de Q_u associado ao seu maior autovalor.

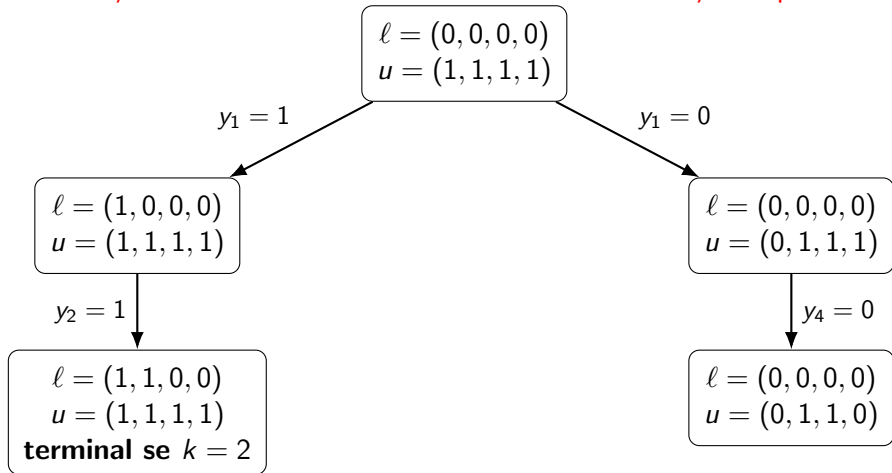
- Resolvemos via Método da Potência Inversa (muito rápido, vamos aprender este método).
- Não precisamos da fatoração espectral completa de Q_u .

- Nó terminal: a partir do qual não precisamos ramificar, pois já enumeramos o suficiente e resta no máximo um suporte possível.
- Função para determinar se é ou não terminal:

$$\text{isTerminal}(\ell, u, k) = \begin{cases} \text{true}, & \sum_i \ell_i \geq k, \\ \text{true}, & \sum_i u_i \leq k, \\ \text{false}, & \text{caso contrário.} \end{cases}$$

- Se terminal $\Rightarrow$ resolve-se exatamente com PCA na submatriz Q_u do suporte.
- Na prática, podaremos o nó pai antes de chegamos a um nó filho para o qual $\sum_i l_i > k$ ou $\sum_i u_i < k$.

Atenção: a árvore B&B não indica todas as ramificações aqui.



- Quando (l, u) não é terminal, o suporte não está totalmente determinado.
- Podemos usar o PCA clássico

$$\max_{x, \|x\|_2=1} x_u^T Q_u x_u$$

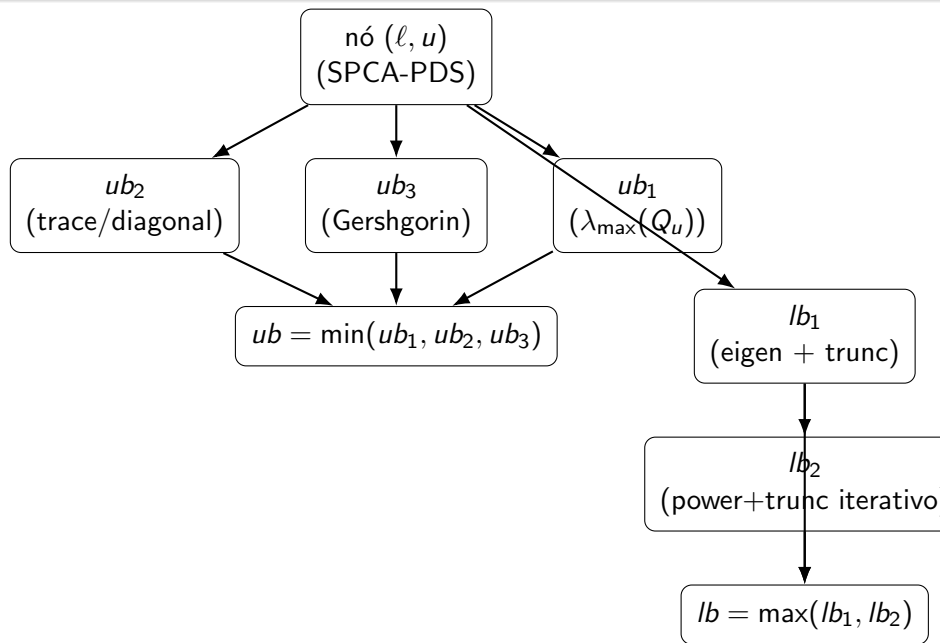
para limitar superiormente o valor ótimo do subproblema enraizado em (l, u) . Neste caso, a matriz Q_u é obtida removendo-se as linhas e colunas para as quais $l_i = 0$.

- Seja $ub_1 = \lambda_{\max}(Q_u)$ este upper bound.
- Este é um lower bound computacionalmente caro. Vamos usá-lo em combinação com outros upper bounds.
- Recorde-se que se $l = i$ este upper bound seria também um lower bound válido e por isso ocorreria a poda do nó.

Dado um vetor $s \in \mathbb{R}^n$, a função

$\text{Trunc}(s, \ell, u, k)$ seleciona exatamente k índices, incluindo os obrigatórios.

- $I_1 = \{i : \ell_i = 1\}$ (obrigatórios)
- $I_2 =$ os $k - |I_1|$ maiores $|s_i|$ dentre os livres ($\ell_i = 0, u_i = 1$)
- suporte final $I = I_1 \cup I_2$
- Com este suporte I , podemos obter um limitante primal.



Bounds no B&B: o que é $ub(\ell, u, k)$?

Em um nó do B&B, temos restrições no suporte via (ℓ, u) . Defina o conjunto de índices ainda permitidos:

$$S(u) = \{ i : u_i = 1 \}.$$

O subproblema do nó (ignorando detalhes) busca um vetor x com $\|x\|_2 = 1$ e suporte viável (tamanho $\leq k$) dentro de $S(u)$.

Objetivo do upper bound

Construir um número $ub(\ell, u, k)$ tal que

$$\max_{\substack{\|x\|_2=1, \|x\|_0 \leq k \\ \text{supp}(x) \subseteq S(u) \text{ e respeita } \ell}} x^\top Q x \leq ub(\ell, u, k).$$

No paper: $ub(\ell, u, k) = \min\{ub_1, ub_2, ub_3\}$.

ub_1 ignora a restrição $\|x\|_0 \leq k$, portanto é um upper bound.

- Construa a matriz Q_u restringindo Q aos índices permitidos $S(u)$:

$$Q_u := Q_{S(u), S(u)} \in \mathbb{R}^{|S(u)| \times |S(u)|}.$$

- Definição:

$$ub_1(\ell, u, k) := \lambda_{\max}(Q_u).$$

- Por que é válido? Para qualquer x com $\text{supp}(x) \subseteq S(u)$, escreva x_S como o vetor de suas coordenadas em $S(u)$. Então:

$$x^\top Q x = x_S^\top Q_u x_S.$$

Pelo quociente de Rayleigh:

$$x_S^\top Q_u x_S \leq \lambda_{\max}(Q_u) \|x_S\|_2^2.$$

Como $\|x\|_2 = 1$ e x não tem massa fora de $S(u)$, temos $\|x_S\|_2^2 = 1$. Logo,

$$x^\top Q x \leq \lambda_{\max}(Q_u) = ub_1.$$

Seja T um suporte viável (tamanho $|T| = k$) respeitando (ℓ, u) . Considere a submatriz principal $Q_T := Q_{T,T}$.

- Se $Q_T \succeq 0$, então seu autovalor máximo é no máximo o traço.

$$\lambda_{\max}(Q_T) \leq \text{tr}(Q_T).$$

Veja: $\text{tr}(Q_T) = \sum_{i=1}^k \lambda_i(Q_T) \geq \lambda_{\max}(Q_T)$.

- Como produzir um upper bound para SPCA a partir desta ideia ? Para qualquer x com suporte T e $\|x\|_2 = 1$:

$$x^\top Q x = x_T^\top Q_T x_T \leq \lambda_{\max}(Q_T) \leq \text{tr}(Q_T).$$

Mas $\text{tr}(Q_T) = \sum_{i \in T} Q_{ii}$. Logo, o melhor que qualquer suporte viável pode fazer é limitado por:

$$ub_2(\ell, u, k) := \max_{\substack{T \text{ viável} \\ |T|=k}} \sum_{i \in T} Q_{ii}.$$

- Na prática: escolha obrigatórios ($\ell_i = 1$) e complete com as maiores diagonais entre os livres ($\ell_i = 0, u_i = 1$).

Teorema de Gershgorin (Resultado Clássico de Álgebra Linear)

Para qualquer matriz simétrica A , todo autovalor λ satisfaz

$$\lambda \leq \max_i \left\{ A_{ii} + \sum_{j \neq i} |A_{ij}| \right\}.$$

Em particular,

$$\lambda_{\max}(A) \leq \max_i \left\{ A_{ii} + \sum_{j \neq i} |A_{ij}| \right\}.$$

Agora aplique a uma submatriz Q_T (suporte T , $|T| = k$):

$$\lambda_{\max}(Q_T) \leq \max_{i \in T} \left\{ Q_{ii} + \sum_{\substack{j \in T \\ j \neq i}} |Q_{ij}| \right\} = \max_{i \in T} \sum_{j \in T} |Q_{ij}|.$$

Como produzir um upper bound a partir desta ideia, dado um suporte T

Para cada coluna/linha candidata i permitida (com $u_i = 1$), maximize a soma de k termos em módulo respeitando (ℓ, u) :

$$ub_3(\ell, u, k) := \max_{i: u_i=1} \max_{\substack{T \text{ viável} \\ |T|=k, i \in T}} \sum_{j \in T} |Q_{ij}|.$$

Na prática: para cada i , inclua obrigatórios e complete com os maiores $|Q_{ij}|$ entre índices livres para totalizar k .

Justificativa: $\lambda_{\max}$ não pode exceder o maior “raio de Gershgorin”; restringir a k entradas fornece um upper bound compatível com esparsidade.

Considere a matriz simétrica

$$Q = \begin{bmatrix} 4 & 1 & 0 & 0 & 0 & 0 \\ 1 & 3 & 1 & 0 & 0 & 0 \\ 0 & 1 & 3 & 1 & 0 & 0 \\ 0 & 0 & 1 & 3 & 1 & 0 \\ 0 & 0 & 0 & 1 & 3 & 1 \\ 0 & 0 & 0 & 0 & 1 & 2.5 \end{bmatrix}.$$

- Q é **definida positiva** (por ex., é estritamente diagonalmente dominante: $Q_{ii} > \sum_{j \neq i} |Q_{ij}|$ em todas as linhas).
- Vamos calcular os bounds do paper para um nó sem restrições adicionais de suporte ($u = 1$, $\ell = 0$), com **esparsidade** $k = 3$.

Recordando que...

Buscamos um upper bound para

$$\max_{\|x\|_2=1, \|x\|_0 \leq 3} x^\top Q x.$$

Cálculo de ub_2 : bound por traço/diagonal (com $k = 3$)

Para qualquer suporte T com $|T| = k$ e qualquer x com $\text{supp}(x) \subseteq T$ e $\|x\|_2 = 1$:

$$x^\top Q x = x_T^\top Q_T x_T \leq \lambda_{\max}(Q_T) \leq \text{tr}(Q_T).$$

Como $\text{tr}(Q_T) = \sum_{i \in T} Q_{ii}$, temos

$$ub_2(k) = \max_{|T|=k} \sum_{i \in T} Q_{ii}.$$

Passo 1: pegue a diagonal de Q

$$\text{diag}(Q) = (4, 3, 3, 3, 3, 2.5).$$

Passo 2: escolha os $k = 3$ maiores valores da diagonal

Os 3 maiores são: 4, 3, 3 (por exemplo, índices $T^* = \{1, 2, 3\}$). Logo,

$$ub_2(3) = 4 + 3 + 3 = 10.$$

Interpretação: nenhum componente 3-esparso pode explicar mais do que a soma das 3 maiores variâncias individuais.

Cálculo de ub_3 : bound tipo Gershgorin (com $k = 3$)

Pelo teorema de Gershgorin, para qualquer submatriz Q_T :

$$\lambda_{\max}(Q_T) \leq \max_{i \in T} \sum_{j \in T} |Q_{ij}|.$$

Para evitar enumerar todos os suportes T com $|T| = 3$, o bound ub_3 faz:

$$ub_3(3) = \max_{i \in \{1, \dots, 6\}} \sum_{j \in T_i} |Q_{ij}|,$$

onde T_i é o conjunto de $k = 3$ índices que maximiza a soma na linha/coluna i (i.e., pegue os 3 maiores valores de $|Q_{ij}|$).

Somas das 3 maiores entradas (em módulo) por coluna/linha

i	3 maiores de $ Q_{ij} $	$\sum$
1	4, 1, 0	5
2	3, 1, 1	5
3	3, 1, 1	5
4	3, 1, 1	5
5	3, 1, 1	5
6	2.5, 1, 0	3.5

Para $k = 3$ e sem restrições adicionais ($u = 1$):

$$ub_2(3) = 10, \quad ub_3(3) = 5.$$

Logo o upper bound usado no nó seria

$$ub = \min\{ub_1, ub_2, ub_3\} \leq \min\{ub_1, 10, 5\} = \min\{ub_1, 5\}.$$

Análise dos limitantes duais (upper bounds)

- ub_2 é **extremamente barato**, mas pode ser bastante fraco quando a diagonal é grande.
- ub_3 ainda é barato e captura um pouco das **correlações** via $|Q_{ij}|$, fornecendo um bound mais apertado no exemplo.

Três upper bounds, um $ub = \min\{ub_1, ub_2, ub_3\}$

- $ub_1 = \lambda_{\max}(Q_u)$: forte mas caro (método da potência).
- ub_2 (diagonal/traço): muito barato; bom quando a diagonal domina e Q é bem “variância individual”.
- ub_3 (Gershgorin com k maiores $|Q_{ij}|$): barato e mais sensível a correlações fora da diagonal.

Composição de bounds

Todos são upper bounds porque relaxam (de modos diferentes) o problema original:

- ub_1 : relaxa a cardinalidade (mantém apenas o conjunto permitido por u).
- ub_2 : limita $\lambda_{\max}(Q_T)$ por $\text{tr}(Q_T)$ e maximiza só a diagonal.
- ub_3 : limita $\lambda_{\max}(Q_T)$ por Gershgorin e escolhe as k maiores contribuições em módulo.

Definições:

- $X(\ell, u, k)$ é o conjunto de todas as soluções viáveis para o nó (l, u) (garantindo a esparsidade desejada).
- $\text{vmax}(Q_u)$ é o autovetor de Q_u associado ao maior autovalor de Q_u .

lb_1 : eigen + trunc + PCA no suporte

- 1 compute $v = \text{vmax}(Q_u)$
- 2 suporte $y = \text{supp}(\text{Trunc}(v, \ell, u, k))$
- 3 $lb_1 = \lambda_{\max}(Q_y)$ (PCA na submatriz do suporte)

lb_2 : iteração power+trunc (Yuan & Zhang adaptado)

Inicialize $lb_2 = 0$, $x^{(1)} \in X(\ell, u, k)$, repita até convergir:

$$x \leftarrow Qx, \quad x \leftarrow \text{Trunc}(x, \ell, u, k), \quad x \leftarrow x / \|x\|_2$$

$$lb_2 = \max\{lb_2, x^\top Qx\}$$

Queremos produzir rapidamente uma solução **viável** para SPCA:

$$\max x^T Qx \quad \text{s.a.} \quad \|x\|_2 = 1, \|x\|_0 \leq k.$$

Uma solução viável dá um **lower bound** (bound primal) para o ótimo.

Ideia do algoritmo

- **Power step:** $y \leftarrow Qx$ (como no método da potência).
- **Truncation:** mantenha apenas k entradas (maiores em módulo) de y e zere o resto.
- **Normalize:** $x \leftarrow y/\|y\|_2$.

O que cada ingrediente faz: o power step “puxa” x na direção de grande variância; a truncagem força a esparsidade.

Operador de truncamento

Denote por $\text{Trunc}_k(v)$ o vetor obtido mantendo as k maiores entradas de $|v|$ e zerando as demais.

Iteração

Escolha $x^{(0)}$ com $\|x^{(0)}\|_2 = 1$ e $\|x^{(0)}\|_0 \leq k$. Para $t = 0, 1, 2, \dots$:

$$\begin{aligned}y^{(t+1)} &\leftarrow Qx^{(t)}, \\ \tilde{y}^{(t+1)} &\leftarrow \text{Trunc}_k(y^{(t+1)}), \\ x^{(t+1)} &\leftarrow \frac{\tilde{y}^{(t+1)}}{\|\tilde{y}^{(t+1)}\|_2}.\end{aligned}$$

Ao final, retorne $x^{(t)}$ e o valor

$$lb_2 := (x^{(t)})^\top Qx^{(t)}.$$

Páre quando $\|x^{(t+1)} - x^{(t)}\|_2$ for pequeno, ou após um número fixo de iterações.

Viabilidade é garantida por construção

Em toda iteração:

- $\text{Trunc}_k(\cdot)$ produz um vetor com $\leq k$ entradas não nulas $\Rightarrow \|x^{(t)}\|_0 \leq k$.
- A normalização força $\|x^{(t)}\|_2 = 1$.

Logo $x^{(t)}$ é sempre viável para SPCA $\Rightarrow (x^{(t)})^\top Q x^{(t)}$ é sempre um **lower bound** do ótimo.

Interpretação geométrica

O algoritmo alterna entre:

- um passo na direção do gradiente de $x^\top Q x$ (pois $\nabla(x^\top Q x) = 2Qx$),
- uma projeção aproximada (gulosa) no conjunto de vetores k -esparsos (via truncamento).

Fatos que são tipicamente verdadeiros (e usados na prática)

- **Não** há garantia geral de atingir o ótimo global.
- Em muitas instâncias, observa-se que os valores $f(x^{(t)}) = (x^{(t)})^\top Q x^{(t)}$ **crecem** até estabilizar.
- O algoritmo tende a convergir para um **ponto fixo** da atualização (um ótimo local/estacionário no suporte escolhido).

Por que a sequência tende a estabilizar?

- Há apenas um número finito de suportes de tamanho k .
- Quando o suporte para de mudar, a iteração vira essencialmente um **método da potência** em uma submatriz Q_T , que converge para o autovetor principal de Q_T (desde que o maior autovalor seja simples, isto é, mult. algébrica igual a 1).

Em resumo: convergência global não é garantida; convergência a ponto fixo/ótimo local é o comportamento esperado.

Suponha que a partir de algum passo t_0 o algoritmo seleciona sempre o mesmo suporte T (com $|T| = k$). Então, para $t \geq t_0$:

$$x_{T^c}^{(t)} = 0, \quad x_T^{(t+1)} = \frac{Q_T x_T^{(t)}}{\|Q_T x_T^{(t)}\|_2}.$$

Consequência

A dinâmica em T é o método da potência aplicado à matriz simétrica Q_T . Logo:

- se $\lambda_{\max}(Q_T)$ é simples, $\text{span}\{x_T^{(t)}\}$ converge o auto espaço do autovetor principal de Q_T ;
- o valor $x^{(t)\top} Q x^{(t)} = x_T^{(t)\top} Q_T x_T^{(t)}$ converge para $\lambda_{\max}(Q_T)$.

Essa é a principal justificativa algébrica para a “boa performance” do lb_2 : ele encontra rapidamente um suporte promissor e faz PCA nele.

- **Inicialização:** começar com $\text{Trunc}_k(v_{\max}(Q))$ (autovetor principal de Q truncado).
- **Parada:** poucas iterações (ex.: 5–20) já dão um LB forte; o B&B cuida da certificação.
- **Empates:** se houver empates nas k maiores entradas, use um critério determinístico (índice).
- **Reinícios:** opcionalmente, rode com 2–3 inicializações para melhorar a chance de achar bom suporte.

Em resumo

lb_2 é uma heurística rápida e simples que produz um bound primal bom; o Optimal-SPCA usa B&B para *certificar* a melhor solução.

Optimal-SPCA (visão resumida)

```
Inicialize nó raiz  $(\ell, u) = (0, 1)$  e fila  $\mathcal{N} = \{(\ell, u)\}$ .  
 $lb \leftarrow 0$ ,  $ub \leftarrow \lambda_{\max}(Q)$ , melhor solução  $\hat{x} \leftarrow 0$ .  
while  $ub - lb > \varepsilon$  do  
  Selecione nó  $(\ell, u) \in \mathcal{N}$  (ex.: maior  $ub$ ).  
  Escolha índice livre  $i$  com  $\ell_i = 0$  e  $u_i = 1$  (heurística).  
  for  $val \in \{0, 1\}$  do  
    Crie filho fixando  $y_i = val$  (atualize  $\ell$  ou  $u$ ).  
    if nó terminal then  
      Resolva exatamente (PCA no suporte):  $upper \leftarrow lower$ .  
    else  
      Calcule  $upper$  e  $lower$  (bounds).  
    end if  
    if  $lower > lb$  then  $lb \leftarrow lower$ ;  $\hat{x} \leftarrow x$   
    end if  
    Pode se  $upper \leq lb$ ; senão insira o filho.  
  end for  
  Atualize o melhor upper bound na lista de nós:  $ub \leftarrow \max\{upper(n) : n \in \mathcal{N}\}$ .  
end while  
return  $\hat{x}$ 
```

- Seleção do índice para branch: priorizar $|v_{\max}(Q_u)_i|$ (aproximar com poucas iterações do power method).
- Cálculo de $\lambda_{\max}$: método da Potência Inversa e parar antecipadamente caso já exceda $\min(ub_2, ub_3)$.
- Manter fila limitada, estratégia híbrida de busca: mistura best-first e depth-first.

O B&B do Optimal-SPCA ramifica sobre as variáveis binárias $y_i \in \{0, 1\}$ que indicam se a coordenada i pertence ao suporte do componente esparso.

Nó (ℓ, u) e índices livres

$$\ell_i = 1 \Rightarrow y_i = 1 \text{ (forçado a entrar)}, \quad u_i = 0 \Rightarrow y_i = 0 \text{ (forçado a sair)}.$$

Os índices ainda indecisos no nó são:

$$\mathcal{F} = \{i : \ell_i = 0, u_i = 1\}.$$

Branching

Escolhido um índice $i^* \in \mathcal{F}$, criam-se dois filhos:

$$\text{Filho 1: } y_{i^*} = 1 \Rightarrow \ell_{i^*} \leftarrow 1, \quad \text{Filho 0: } y_{i^*} = 0 \Rightarrow u_{i^*} \leftarrow 0.$$

Como escolher i^* : regra baseada no relaxamento (autovetor)

A escolha do índice de branching usa informação do **relaxamento** que gera o bound ub_1 .

Passos

- 1 Defina o conjunto permitido $S(u) = \{i : u_i = 1\}$ e a submatriz

$$Q_u := Q_{S(u), S(u)}.$$

- 2 Calcule (ou aproxime) o autovetor principal $v_{\max}(Q_u)$.
- 3 Selecione para branching o índice livre com maior magnitude no autovetor:

$$i^* \in \arg \max_{i \in \mathcal{F}} |v_i|.$$

Interpretação

O relaxamento contínuo sugere uma direção ótima v . Se $|v_i|$ é grande, a coordenada i tende a ser importante para obter alto $x^\top Qx$; ramificar nela cedo normalmente reduz a ambiguidade do suporte e fortalece a poda.

Aproximação barata de $v_{\max}(Q_u)$

Não é necessário computar $v_{\max}$ exatamente:

- use poucas iterações do método da potência:

$$v \leftarrow \frac{Q_u v}{\|Q_u v\|_2},$$

- 5–15 iterações já costumam ordenar bem os $|v_i|$.

Isso reduz custo por nó sem perder muito em qualidade de branching.

Empates e estabilidade

- Se houver empates em $|v_i|$, escolha um critério determinístico (menor índice).
- Alternativa (opcional): escolher entre os top- r índices o que mais melhora um bound barato (ex.: via ub_2 ou ub_3).

Branching eficiente depende mais de uma boa *ordenação aproximada* do que de um *autovetor exato*.

Abordagem sequencial: Encontrado o primeiro PCA esparsos z , o que fazer ?

- Seja $z \in \arg \max \{x^T Q x : \|x\|_2 = 1, \|x\|_0 \leq k\}$.
- Projetamos as linhas e colunas de Q em $\text{span}\{z\}^\perp$:
 $\hat{Q} = (I - zz^T)Q(I - zz^T)$.
- Resolvemos $w \in \arg \max \{x^T \hat{Q} x : \|x\|_2 = 1, \|x\|_0 \leq k\}$.
- Não há garantia de que w e z sejam perpendiculares.
- w, z não são necessariamente uma solução ótima de PCAs k esparsos para o problema PCA multi-vetor.
- A abordagem sequencial não garante estas propriedades.
- Se o posto de Q é m então são necessários mais de m vetores para explicar a variância total $\|Q\|_F^2$.